LeRobot Dataset Inspector (In Bash)

Objective

Write a Bash tool that takes a single LeRobot dataset **or a batch (folder of many datasets)** and, for each one, reports its statistics, verifies its integrity against the actual files on disk, and flags anything wrong.

I will hand you datasets that contain deliberate problems, missing files, missing metadata fields, and incorrect parameters. Your tool must detect and report each one precisely. A tool that crashes, or that reports "PASS" on a broken dataset, fails the assignment.

Questions your tool must answer (per dataset)

1. Total recorded duration in hours (derive it, it is not stored directly).
2. Number of episodes.
3. FPS.
4. Number of cameras, their names, and resolution.
5. Robot type and the list of tasks.
6. On-disk size.

Integrity checks your tool must perform

7. **Per-episode, cross-modal consistency:** the parquet row count, the episode `length` in `meta/episodes.jsonl`, and the frame count of each camera `.mp4` (read it with `ffprobe`, don't trust the filename) must all agree. Report every mismatch localized to the exact episode and file.
8. **File accounting:** parse the `data_path` / `video_path` templates and `chunks_size` in `meta/info.json`, compute the exact files that should exist, and report **missing files** (referenced, absent) and **orphan files** (present, not referenced). Get the chunk math right.
9. **Metadata consistency:** `total_episodes`, `total_frames`, `total_videos`, `total_tasks` must match what is actually on disk and in the `.jsonl` files. Detect missing or empty fields, and zero/invalid values.
10. **Temporal integrity:** read the `timestamp` column, confirm it increases monotonically, compute the empirical fps and compare it to the declared fps, and report dropped frames / time gaps.
11. **Statistical validation:** recompute mean/std/min/max for `action` and `observation.state` from the parquet data and diff against the stored stats in `meta/stats.json` / `episodes_stats.jsonl` within a configurable tolerance.

12. **Version awareness:** detect codebase_version and handle the schema differences between versions (v2.0 / v2.1 / older).
13. **Batch mode:** process hundreds of datasets in parallel with a configurable concurrency limit, and flag cross-dataset anomalies (e.g. one dataset's fps or episode length disagreeing with the rest).
14. **Verdict:** a clear PASS/FAIL and issue list per dataset, plus a roll-up summary (total datasets, episodes, hours, how many passed).

Engineering demands

- Runs as ./lerobot-inspect after chmod +x; proper shebang; -h/--help.
- Accepts paths as arguments; one dataset and a batch use the same code path.
- set -euo pipefail, all variables quoted, and **shellcheck-clean at zero warnings** (I will run it).
- **Read-only**, never modifies, moves, or writes into a dataset.
- Two outputs: a human-readable report and a valid **JSON report** (--json).
- **Documented exit codes** (ok / warnings / integrity failure / usage error) and a --strict flag that turns warnings into failures.
- Clear, precise error handling: malformed JSON, truncated parquet, missing meta/, zero-byte video, missing dependency, each gives an exact message and correct exit code, never a raw error dump or a false PASS.
- No hardcoded paths, dataset names, or magic numbers; tolerances are configurable.
- Logs to stderr, data to stdout, so it composes in a pipe.
- Bash has no floating-point math, handle it correctly.

Deliverables

1. The tool or the script.
2. A test that generates broken datasets (drop a frame, truncate a parquet, corrupt info.json, delete a chunk, perturb a stat) and shows your tool catches each.
3. A README: usage, every flag, dependencies, the JSON schema, and the exit-code table.
4. A short DESIGN note (≤1 page): assumptions, tradeoffs, and what you'd improve.

Evaluation

(1) correctness against a dataset I check by hand;
(2) does it actually catch the corruption I put inside;
(3) robustness and batch performance;
(4) shellcheck-clean, readable code.
In a ~30-minute review I will run it on a fresh dataset and a broken one, ask you to explain specific lines, and ask you to add one new check on the spot.

Rules

Use references and tools to learn, but every line you submit must be one you understand and can defend and modify under questioning. Ask sharp questions early; don't guess silently. Send me the deliverables and block 30 minutes for the walkthrough.